Guía de estudio y ayuda memoria del código (`src`)

Documento vivo: **se actualiza al cerrar cada fase**. Sirve para buscar rápido qué hay en cada carpeta y en cada archivo, y para qué sirve.

Última actualización: 19/10/2026, al cierre de la Fase 1 (las Fases 2 a 5 todavía no tienen código).

Convención de rutas: todo está bajo `src/main/java/ar/edu/iua/iw3/`, salvo que se diga otra cosa.

Columna **Origen**: *Cátedra* = ya venía del proyecto de clase · *F1* = creado o cambiado en la Fase 1.

---

1. El proyecto en un párrafo

Es un backend (API REST, sin pantallas) que administra **órdenes de carga de gas líquido**. Tres sistemas externos le hablan por HTTP:

- **SAP** manda la orden: camión, chofer, cliente, producto, fecha prevista y cuántos kg cargar (*preset*).
- **La balanza (TMS)** manda el peso del camión vacío (*tara*) y, al final, el peso final.
- **El sistema de carga** pide autorización con una contraseña y manda en tiempo real los datos del caudalímetro.

Con todo eso se calcula la **conciliación**: balanza contra caudalímetro.

Los estados de la orden son secuenciales, nunca vuelven atrás:

Estado	Significado
1	Pendiente de pesaje inicial
2	Con pesaje inicial registrado (se carga acá)
3	Cerrada para carga
4	Finalizada

2. Qué se hizo en cada fase

Fase	Contenido	Estado
0	Auditoría y línea base (no se tocó código)	Hecha
1	**Punto 1:** SAP envía la orden y se crea en estado 1	Hecha
2	**Punto 2:** la balanza envía la tara, se genera la contraseña de 5 dígitos y pasa a estado 2	Pendiente
3	**Punto 3:** activación con contraseña y datos del caudalímetro en tiempo real	Pendiente
4	**Punto 4:** cerrar la orden (estado 3)	Pendiente
5	**Punto 5:** pesaje final, estado 4 y conciliación	Pendiente

3. Para qué sirve cada carpeta (lo que hay que recordar)

Cada carpeta responde **una pregunta**. Esa es la regla para saber dónde va un archivo nuevo.

Carpeta	Pregunta que responde	Qué NO debe tener
`model`	¿Qué datos hay y cómo se guardan?	Reglas, HTTP
`model/persistence`	¿Cómo se buscan y guardan?	Reglas, HTTP
`business`	¿Qué reglas se aplican?	HTTP, SQL
`business/exception`	¿Qué errores de negocio existen?	—

Carpeta	Pregunta que responde	Qué NO debe tener
`controllers`	¿Cómo entra y sale el pedido por HTTP?	Reglas
`integration`	¿Qué sistema externo llama?	Reglas
`security`	¿Qué se puede hacer sin token?	Lógica del dominio
`auth`	¿Quién es el usuario?	Lógica del dominio
`config`	¿Cómo se arma la aplicación?	Lógica del dominio
`util`	¿Qué herramientas se comparten?	Cosas de una sola capa

Camino de un pedido: `security` / `auth` (revisan el token) → `integration` o `controllers` (lo reciben) → `business` (decide) → `model/persistence` (guarda) → `model` (las tablas).

Regla para decidir dónde va algo nuevo: ¿qué cambiaría si esto cambiara? Si cambia la URL o el código de respuesta, `controllers`. Si cambia una regla, `business`. Si cambia una tabla, `model`.

---

4. `model/`: las fichas de datos

Cada ficha es una clase que se convierte en una tabla de la base de datos.

Archivo	Tabla	Qué es y para qué sirve	Origen
`Order.java`	`orders`	La orden de carga. Une camión, chofer, cliente y producto. Guarda número (único), fecha prevista, preset, estado y fecha de recepción. Se llama `orders` porque `ORDER` es palabra reservada de SQL	F1
`OrderStatus.java`	—	Los 4 estados. En el JSON se publica el número (1 a 4)	F1
`OrderStatusConverter.java`	—	Traduce estado ↔ número para guardar 1 a 4 en la base, y no 0 a 3	F1
`OrderStatusLog.java`	`order_status_logs`	Historial: estado anterior (`null` al crear), estado nuevo, fecha, quién lo hizo y observaciones	F1
`Truck.java`	`trucks`	Camión: código externo (único), patente, descripción y lista de cisternas	F1
`TruckTank.java`	`truck_tanks`	Una cisterna: número y capacidad en litros. Una fila por cisterna	F1
`Driver.java`	`drivers`	Chofer: código externo, nombre, apellido y documento (opcional)	F1
`Customer.java`	`customers`	Cliente: código externo, razón social y contacto (opcional)	F1
`Product.java`	`gas_products`	Producto de gas: código externo, nombre y descripción (opcional). Usa `gas_products` porque `products` ya existe en bases viejas con otro esquema	F1 (reemplazó al del demo)

Idea clave: camión, chofer, cliente y producto se identifican por su **código externo** (el que usa SAP), nunca por el `id` interno.

`model/persistence/`: los repositorios

Cada uno busca y guarda un tipo de ficha. Son interfaces: Spring arma el código.

Archivo	Qué hace	Origen
`OrderRepository.java`	Busca una orden por número y lista las órdenes paginadas, de la más reciente a la más antigua	F1
`OrderStatusLogRepository.java`	Trae el historial de una orden, del más antiguo al más reciente	F1
`TruckRepository.java`	Busca un camión por código externo	F1
`DriverRepository.java`	Busca un chofer por código externo	F1

Archivo	Qué hace	Origen
`CustomerRepository.java`	Busca un cliente por código externo	F1
`ProductRepository.java`	Busca un producto por código externo	F1 (reemplazó al del demo)

---

5. `business/`: las reglas

Archivo	Qué es	Origen
`IOrderBusiness.java`	**La lista** de operaciones sobre órdenes (el "menú"). No tiene reglas	F1
`OrderBusiness.java`	**Las reglas de verdad** (la "cocina")	F1
`OrderValidator.java`	Revisa que la orden recibida esté completa y con valores lógicos	F1

`IOrderBusiness` frente a `OrderBusiness`

	`IOrderBusiness`	`OrderBusiness`
Qué tiene	Solo los nombres de las operaciones	Las operaciones con su código
Reglas	Ninguna	Todas

Los controllers piden `IOrderBusiness` y Spring les entrega `OrderBusiness`. Así los controllers no dependen de cómo está escrita la regla.

Operaciones de `IOrderBusiness` hoy: `list(Pageable)`, `load(orderNumber)`, `listStatusLog(orderNumber)`, `add(order, actor)`.

Qué hace `OrderBusiness.add` (paso a paso)

1. Valida con `OrderValidator`. Si hay errores, los devuelve **todos juntos** (400).
2. Revisa que el número de orden no exista (si existe, 302).
3. Sincroniza camión, chofer, cliente y producto por código externo: si existen los actualiza, si no los crea. Las cisternas solo se reemplazan si cambiaron.
4. Crea la orden en **estado 1** y le pone la fecha de recepción. El cliente no puede fijar ni el estado ni el `id`: se copian solo los campos permitidos.
5. Anota el historial: de nada a estado 1, con el usuario del token.
6. Todo ocurre **en una transacción**: si algo falla, no queda ninguna fila a medias.

Qué revisa `OrderValidator`

- `orderNumber` entero mayor que 0; fecha prevista presente; `preset` mayor que 0.
- Camión: código y patente; al menos una cisterna; cada cisterna con número y litros mayores que 0; sin números repetidos.
- Chofer: código, nombre y apellido.
- Cliente: código y razón social.
- Producto: código y nombre.
- Los textos no pueden pasar el largo de su columna.

`business/exception/`: errores de negocio

Archivo	Se convierte en	Origen
`NotFoundException.java`	404 (no se encontró)	Cátedra
`FoundException.java`	**302** (ya existe; convención de la cátedra para duplicados)	Cátedra

Archivo	Se convierte en	Origen
`BusinessException.java`	500 (falla interna)	Cátedra
`BadRequestException.java`	400 (datos inválidos)	**F1**

6. `controllers/`: la capa HTTP

No tienen reglas: reciben el pedido, llaman al negocio y devuelven el código.

Archivo	Qué hace	Origen
`OrderRestController.java`	Consultas: lista paginada, una orden por número, historial de estados	F1
`BaseRestController.java`	Clase madre de los controllers. Tiene `getUserLogged()`: el usuario del token, de donde sale el "quién" del historial	Cátedra + F1
`Constants.java`	Todas las URLs en un solo lugar	Cátedra + F1

7. `integration/`: un subpaquete por sistema externo

Hoy solo SAP. Después irán `tms/` (balanza) y `loader/` (sistema de carga).

Archivo	Qué hace	Origen
`sap/controllers/OrderSapRestController.java`	La puerta de SAP. Recibe la orden, la pasa al negocio y responde 201 (con header `location`), 302, 400 o 500	F1

8. `security/` y `auth/`

`security` define qué es público. `auth` maneja usuarios, login y token. Casi todo viene de la cátedra.

Archivo	Qué hace	Origen
`security/SecurityConfiguration.java`	Rutas públicas y reglas. Todo exige token salvo login, `/error`, Swagger y `/demo/**`. Define la encriptación BCrypt y la sesión sin estado. Se le permitió `/error` para que los errores de formato no salgan como 401 mudos	Cátedra + F1
`auth/User.java`	Usuario (`users`). Valida cuenta vencida, bloqueada o deshabilitada. Tiene sus roles	Cátedra
`auth/Role.java`	Rol (`roles`)	Cátedra
`auth/UserRepository.java`	Busca un usuario por nombre o email	Cátedra
`auth/RoleRepository.java`	Acceso a la tabla de roles	Cátedra
`auth/IUserBusiness.java` y `UserBusiness.java`	Cargar usuario, cambiar contraseña, habilitar/deshabilitar, listar	Cátedra
`auth/BadPasswordException.java`	Error de contraseña incorrecta	Cátedra
`auth/custom/CustomAuthenticationManager.java`	Valida usuario, estado de cuenta y contraseña (BCrypt)	Cátedra
`auth/controller/AuthRestController.java`	`POST /api/v1/login`: emite el token. También tiene `GET /demo/encodepass` (devuelve el hash BCrypt de una contraseña; sirve para crear usuarios de prueba). Se le quitó el `produces=text/plain` para que el login fallido devuelva su mensaje	Cátedra + F1

Archivo	Qué hace	Origen
`auth/filters/AuthConstants.java`	Constantes del token: clave, vencimiento (1 h), nombre del header y del parámetro	Cátedra
`auth/filters/JWTAuthorizationFilter.java`	En cada pedido lee el token (header `Bearer` o `?authtoken=`) y deja al usuario identificado	Cátedra

Token (JWT): se firma, no se cifra. Vence a la hora. Dura aunque después deshabiliten al usuario, porque no consulta la base.

9. `config/` y `util/`

Archivo	Qué hace	Origen
`config/OpenApiConfig.java`	Configura Swagger: título y botón *Authorize* para pegar el token	F1
`util/StandartResponse.java`	El JSON de error: `message`, `code`, `devInfo`. El nombre con errata viene de la cátedra	Cátedra
`util/IStandartResponseBusiness.java` y `StandartResponseBusiness.java`	`build(status, ex, message)` arma ese JSON	Cátedra

10. Raíz de `src/main/java` y otras carpetas de `src`

Archivo	Qué hace	Origen
`Iw3Application.java`	Arranque de la app. Fija la zona horaria y loguea el perfil activo. Se le sacaron componentes del demo	Cátedra + F1
`src/main/resources/application.properties`	Conexión a la base y configuración general. Incluye la zona horaria de Argentina para la base (`hibernate.jdbc.time_zone`)	Cátedra + F1
`src/main/resources/application-mysqldev.properties` y `-mysqlprod.properties`	Perfiles de desarrollo y producción	Cátedra
`src/test/java/.../Iw3ApplicationTests.java`	Un solo test: que la aplicación arranque (`contextLoads`). Necesita MySQL	Cátedra

11. Endpoints que existen hoy

Método y ruta	Qué hace	Respuestas
`POST /api/v1/login`	Usuario y contraseña (formulario); devuelve el token en texto	200, 401
`POST /api/v1/integration/sap/orders`	**Punto 1:** SAP envía la orden	201 + `location`, 302, 400, 401, 500
`GET /api/v1/orders?page=&size=`	Lista paginada (`size` 20 por defecto, máximo 100). Responde `content` y `page`	200
`GET /api/v1/orders/{n}`	Una orden	200, 404
`GET /api/v1/orders/{n}/status-log`	Historial de estados	200, 404
`/swagger-ui.html`, `/v3/api-docs`	Documentación	públicos
`GET /demo/encodepass?password=`	Hash BCrypt (demo de la cátedra, público; cerrar antes de desplegar)	200

Códigos: **201** creada · **302** ya existe (convención de la cátedra) · **400** datos inválidos · **401** sin token o inválido · **404** no existe · **500** falla interna.

12. Fuera de `src`

Archivo	Para qué sirve
`pom.xml`	Dependencias y build (Maven)
`docker-compose.yml`	La base MySQL (contenedor `iw3-gas-mysql`, puerto 3306, base `iw3_db`)
`CLAUDE.md`	Estado del proyecto y reglas del equipo
`docs/plan-de-trabajo.md`	Plan completo y decisiones
`docs/fase-0-auditoria.md`, `docs/fase-1-recepcion-ordenes.md`	Informe de cada fase y guía de estudio con preguntas del profesor
`docs/postman/`	Colección Postman (10 pedidos, 19 verificaciones)
`docs/sql/seed-dev-users.sql`	Usuario de prueba `admin` (solo desarrollo)
`docs/pdf/`	PDF de los informes y de esta guía

13. Qué se borró en la Fase 1

30 archivos del demo de la cátedra: `integration/cli1`, `integration/cli2`, los `config/profile/*ScanConfig`, `util/JsonUtiles`, productos y categorías de ejemplo, y sus archivos de configuración. Siguen en el historial de git.

14. Decisiones que conviene saber defender

Decisión	Por qué
El `@RequestBody` es la entidad y el negocio copia solo los campos permitidos	Siguiendo a la cátedra, pero evitando que el cliente fije el estado o el `id`
Transacción con `rollbackFor = Exception.class`	Las excepciones de negocio son *checked* y Spring no revierte por ellas
Duplicado = 302	Convención de la cátedra. 409 sería más preciso
Datos inválidos = 400	Extensión nuestra: no es culpa del servidor
Estado guardado como 1 a 4	La consigna habla de estados 1 a 4
Tablas `orders` y `gas_products`	`ORDER` es reservada; `products` ya existe en bases viejas
Validación manual en el negocio	La cátedra no usa Bean Validation
`hibernate.jdbc.time_zone`	Sin eso la base guardaba la hora 3 h adelantada

15. Buscador rápido: "si quiero..., voy a..."

Si quiero...	Voy a
Ver qué datos lleva la orden	`model/Order.java`
Cambiar qué se le exige a SAP	`business/OrderValidator.java`
Cambiar lo que pasa al recibir la orden	`business/OrderBusiness.java` (`add`)
Cambiar un código HTTP	El controller del endpoint (`integration/sap/...` o `controllers/...`)
Agregar una URL	`controllers/Constants.java` y el controller
Abrir o cerrar una ruta sin token	`security/SecurityConfiguration.java`

Si quiero...	Voy a
Ver cómo se arma el token	`auth/controller/AuthRestController.java`
Cambiar el formato de error	`util/StandartResponseBusiness.java`
Ver los estados de la orden	`model/OrderStatus.java`
Cambiar la conexión a la base	`src/main/resources/application.properties`

16. Cómo mantener este documento

Al cerrar cada fase:

1. Agregar a las tablas los archivos nuevos y marcar los cambiados con la fase (F2, F3...).
2. Actualizar la tabla de endpoints (sección 11) y de fases (sección 2).
3. Actualizar la fecha de la cabecera.
4. Regenerar el PDF:

```
node docs/pdf/md2pdf.js docs/guia-estudio-src.md docs/pdf/guia-estudio-src.pdf "Guía de estudio del código"
```